Matching and Linear Programming

Hungarian algorithm, blossom, LP duality & simplex, flows with demands

Learning goals

- Solve the assignment problem in $O(n^2m)$ with the **Hungarian algorithm** and understand its potentials as an LP dual
- Find maximum matchings in **general** graphs with Edmonds' blossom algorithm
- Formulate problems as **linear programs**, take duals, and use complementary slackness
- Run a robust two-phase **simplex** on small LPs
- Model **lower bounds** on flow: feasible circulations, max/min flow with demands

Three problems that bipartite max-flow cannot do

1. **Assignment**, n = 500: n workers, n jobs, cost matrix, minimise total cost
   → MCMF works but is slow and heavy; Hungarian is 30 lines, $O(n^3)$
2. **Pairing**, 500 students, "can share a room" graph, maximise pairs
   → the graph is **not bipartite**: augmenting-path BFS fails on odd cycles
3. **Mixing**, 50 ingredients, 80 linear nutrient constraints, maximise profit
   → continuous variables: a **linear program**

Assignment as an LP

Primal (relaxation is integral: the matrix is totally unimodular)

```
min Σ a_ij x_ij   s.t.  Σ_j x_ij = 1 (each row),  Σ_i x_ij = 1 (each col),  x ≥ 0
```

Dual: potentials u_i (rows), v_j (columns)

```
max Σ u_i + Σ v_j   s.t.   u_i + v_j ≤ a_ij
```

- Complementary slackness: x_ij = 1 ⇒ **u_i + v_j = a_ij** (edge is *tight*)
- Perfect matching on tight edges + feasible potentials ⇒ **both optimal**

Hungarian algorithm: idea

Maintain feasible potentials and a matching that uses only tight edges.

Add rows one at a time (row i):

1. Grow an alternating tree from row i using **reduced costs** $a_{ij} - u_i - v_j \geq 0$
2. This is Dijkstra in disguise: `minv[j]` = cheapest reduced cost to reach column j
3. Pick the unvisited column with smallest `minv`, call it δ
4. Shift potentials by δ (tree rows $+\delta$, tree columns $-\delta$): **feasibility kept, new tight edge appears**
5. Reached a free column → augment along `way[]`

Each row: $O(n \cdot m)$. Total $\mathbf{O(n^2 m)}$.

Hungarian: code (1/2)

```cpp
// a is (n+1) x (m+1), 1-indexed, n <= m. Returns min cost; match[i] = column of row i.
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a) {
    int n = a.size() - 1, m = a[0].size() - 1;
    const ll INF = numeric_limits<ll>::max() / 4;
    vector<ll> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);           // p[j] = row matched to column j
    for (int i = 1; i <= n; i++) {
        p[0] = i;                               // column 0 = virtual root
        int j0 = 0;
        vector<ll> minv(m + 1, INF);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            for (int j = 1; j <= m; j++) if (!used[j]) {
                ll cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
```

Hungarian: code (2/2)

```
            for (int j = 0; j <= m; j++)          // shift potentials by delta
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);                     // stop at a free column
        do {                                      // augment
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> match(n + 1);
    for (int j = 1; j <= m; j++) if (p[j]) match[p[j]] = j;
    return {-v[0], match};                        // -v[0] = optimal cost
}
```

- Maximise? Negate costs. Forbidden pair? Cost = large constant (check it is not used)

Why −v[0] is the answer

- Column 0 is the virtual root of every search, so it is always `used`
- In one step: tree rows (k + 1 of them) get +δ, tree columns (k real + column 0) get −δ
- So the dual objective Σu + Σv over **real** rows/columns grows by δ, and v[0] drops by δ
- At the end potentials are feasible and matched edges tight ⇒ Σu + Σv = optimal cost = **−v[0]**
- In debug builds also recompute Σ a[i][match[i]]

Tested against brute force over all injections for n ≤ 6, m ≤ 7, negative costs included.

General graphs: what breaks

Berge: a matching M is maximum ⇔ no **augmenting path** (alternating, both ends free).

Bipartite BFS labels vertices *even/odd* by distance from a free root and only extends from even vertices.



- BFS from r: b, e odd; c, d even; edge c–d joins two **even** vertices
- b is odd, so b–x is never explored, yet **r–e═d–c═b–x** is augmenting
- Odd alternating cycle (r b c d e) = **blossom**: the path must go around it the other way

Edmonds: shrink the blossom

Lemma. Contract a blossom B (odd cycle of length 2k+1 with base b) into one vertex. G has an augmenting path w.r.t. M ⇔ G/B has one w.r.t. M/B.

- Every vertex of a blossom can be reached with **even** parity (go around the right way)
- So after contraction, all blossom vertices become even and enter the BFS queue
- Implementation trick: never physically contract; keep `base[v]` = base of v's outermost blossom

Blossom: implementation map

Array	Meaning
`match[v]`	partner or −1
`par[v]`	for odd vertices: the even vertex that discovered it
`base[v]`	base of the blossom containing v
`used[v]`	v is even (in queue)

Edge (v, to) seen from even v:

- `to` is even (root or matched-and-labelled) → **blossom**: lca of bases, relabel, push odd vertices
- `to` unlabelled & free → **augmenting path found**
- `to` unlabelled & matched → label odd, push its partner as even

Blossom: code (1/2)

```
int lca(int a, int b) {                          // in the alternating forest
    vector<char> seen(n, false);
    for (;;) { a = base[a]; seen[a] = true; if (match[a] == -1) break; a = par[match[a]]; }
    for (;;) { b = base[b]; if (seen[b]) return b; b = par[match[b]]; }
}
void markPath(int v, int b, int child) {    // walk from v up to blossom base b
    while (base[v] != b) {
        inBlossom[base[v]] = inBlossom[base[match[v]]] = true;
        par[v] = child;                          // reverse pointers so paths go around the cycle
        child = match[v];
        v = par[match[v]];
    }
}
int solve() {                                    // after greedy warm start
    for (int v = 0; v < n; v++) if (match[v] == -1) {
        int x = findPath(v);
        if (x != -1) res++;
        while (x != -1) { int pv = par[x], ppv = match[pv];
                          match[x] = pv; match[pv] = x; x = ppv; }
    }
    return res;
}
```

Blossom: code (2/2) — the BFS

```
int findPath(int root) {
    fill(used.begin(), used.end(), 0); fill(par.begin(), par.end(), -1);
    iota(base.begin(), base.end(), 0);
    used[root] = true; q.assign(1, root);
    for (size_t qh = 0; qh < q.size(); qh++) {
        int v = q[qh];
        for (int to : g[v]) {
            if (base[v] == base[to] || match[v] == to) continue;
            if (to == root || (match[to] != -1 && par[match[to]] != -1)) {
                int cur = lca(v, to);
                fill(inBlossom.begin(), inBlossom.end(), 0);
                markPath(v, cur, to); markPath(to, cur, v);
                for (int i = 0; i < n; i++) if (inBlossom[base[i]]) {
                    base[i] = cur;
                    if (!used[i]) used[i] = true, q.push_back(i);
                }
            } else if (par[to] == -1) {
                par[to] = v;
                if (match[to] == -1) return to;
                used[match[to]] = true; q.push_back(match[to]);
            }
        }
    }
    return -1;
}
```

Blossom: complexity and alternatives

- n searches × O(n) blossoms × O(n) relabel → **O(V³)**, comfortably n ≤ 500, usually n ≤ 2000 sparse
- Greedy initial matching: a large constant-factor win
- **Randomised**: ν(G) = rank(Tutte matrix)/2 with random values mod p, O(V³) Gaussian elimination, one-sided error
- **Weighted** general matching exists (Edmonds + duals, O(V³)) — 200+ lines; keep in the notebook, don't write in contest
- Tested: vs bitmask DP on 3000 random graphs, n ≤ 12

Linear programming

Standard form (what our solver takes):

```
maximise  cᵀx   subject to   A x ≤ b,   x ≥ 0
```

- Equality $a \cdot x = \beta$ → two inequalities
- Free variable x → $x^{+} - x^{-}$
- Minimise → maximise $-c^{T}x$
- ≥ constraint → multiply by -1 (b may become negative → phase 1)

Optimum (if finite) is at a **vertex** of the feasible polytope.

LP duality

Primal (max)	Dual (min)
max $c^T x$	min $b^T y$
$A\,x \le b$	$A^T\,y \ge c$
$x \ge 0$	$y \ge 0$

- **Weak duality**: $c^T x \le b^T y$ for any feasible pair → every dual solution is an upper bound
- **Strong duality**: optima are equal (if either is finite)
- **Complementary slackness**: $y_i > 0 \Rightarrow$ constraint i tight; $x_j > 0 \Rightarrow$ dual constraint j tight

Duality you already know

Primal	Dual
max flow	min cut (as an LP, integral)
max bipartite matching	min vertex cover (König)
min-cost assignment	max Σ potentials (Hungarian)
shortest path s → t	max "height" d_t − d_s with d_v − d_u ≤ w_uv
min-cost flow	potentials / reduced costs

Contest use: if the primal is awkward, write the dual. Often the dual is a shortest path / flow on a graph with few variables. Difference constraints `x_v − x_u ≤ w` are always a shortest-path problem.

Simplex: the tableau idea

- Add slack s_i: $A x + s = b$. Basis = m variables, the rest are 0
- Start: basis = slacks (valid when $b \geq 0$)
- **Pivot**: pick entering column with negative reduced cost, leaving row by **min ratio test**
- Objective never decreases; stop when no negative reduced cost (optimal) or no leaving row (**unbounded**)
- Degenerate pivots can **cycle**: **Bland's rule** (smallest eligible index) provably prevents it; Dantzig's rule (most negative) with index tie-breaks is faster and cycles only in contrived cases

Worst case exponential; in practice ~2–3 m pivots, each $O(mn)$.

Simplex: phase 1

If some b_i < 0 the origin is infeasible.

- Add one artificial variable x_0 to every row: A x − x_0 ≤ b
- Pivot x_0 into the row with the most negative b → all rows feasible
- Minimise x_0 (phase 1 objective). If optimum > 0 → **infeasible**
- Else pivot x_0 out and run phase 2 on the real objective

The reference implementation (KACTL-style tableau) does both phases in ~70 lines.

Simplex: pivot

```
void pivot(int r, int s) {                    // D is (m+2) x (n+2)
    T inv = 1 / D[r][s];
    for (int i = 0; i < m + 2; i++) if (i != r && fabs(D[i][s]) > EPS) {
        T f = D[i][s] * inv;
        for (int j = 0; j < n + 2; j++) D[i][j] -= D[r][j] * f;
        D[i][s] = D[r][s] * f;
    }
    for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] *= inv;
    for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
}
```

- Pivot = Gauss–Jordan step on the tableau, O(mn); swaps basic/non-basic labels
- Row m = phase-2 objective, row m+1 = phase-1 objective

Simplex: main loop

```
bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
        int s = -1;                          // entering: most negative, ties by index
        for (int j = 0; j <= n; j++) if (N[j] != -phase)
            if (s == -1 || make_pair(D[x][j], N[j]) < make_pair(D[x][s], N[s])) s = j;
        if (D[x][s] >= -EPS) return true;
        int r = -1;                          // leaving row: min ratio
        for (int i = 0; i < m; i++) if (D[i][s] > EPS)
            if (r == -1 || make_pair(D[i][n+1] / D[i][s], B[i]) <
                           make_pair(D[r][n+1] / D[r][s], B[r])) r = i;
        if (r == -1) return false;           // unbounded
        pivot(r, s);
    }
}
```

Simplex in practice

- Sizes: m, n $\lesssim$ 200–500 with doubles is typically fine
- `EPS` $\approx$ 1e−9; scale input so numbers are O(1)–O(10^6)
- Integer answers? LP gives fractional vertices unless the matrix is **totally unimodular** (flows, bipartite matching, interval matrices)
- Validation: tested on 3000 random 3-variable LPs vs vertex enumeration (incl. infeasible), and on assignment LPs vs Hungarian
- Codeforces 375E *Red and Black Tree*: intended solution is an LP solved by simplex

Flows with lower bounds

Edge u → v with flow in **[l, r]**:

1. Replace by capacity r − l
2. Pretend l units already flow: `excess[v] += l`, `excess[u] -= l`
3. New source S′ → v with cap excess[v] (> 0); v → T′ with cap −excess[v] (< 0)
4. Feasible circulation ⇔ max-flow S′→T′ **saturates** all S′ edges

```
u ──[l, r]──▶ v      ⇒     u ──(r-l)──▶ v,  S' ─(l)─▶ v,  u ─(l)─▶ T'  (netted per vertex)
```

With a source and sink

- **Feasible s–t flow**: add t → s with capacity ∞, then check circulation
- **Max flow with bounds**: after feasibility, value = flow on t → s; delete that edge; add max-flow s → t in the residual
- **Min flow with bounds**: after feasibility, delete t → s; subtract max-flow **t → s** in the residual
- Actual flow on edge i = l_i + (flow on its reduced edge)

Classic uses: "each row/column sum in [a, b]" matrix rounding, scheduling with minimum staffing, Codeforces 704D.

Bounded flow: code

```
struct BoundedFlow {
    int n; Dinic d; vector<ll> excess; vector<int> ids; vector<ll> lows;
    BoundedFlow(int n) : n(n), d(n + 2), excess(n) {}
    void addEdge(int u, int v, ll lo, ll hi) {
        ids.push_back(d.addEdge(u, v, hi - lo)); lows.push_back(lo);
        excess[v] += lo; excess[u] -= lo;
    }
    bool circulation() {
        int S = n, T = n + 1; ll need = 0;
        for (int v = 0; v < n; v++) {
            if (excess[v] > 0) d.addEdge(S, v, excess[v]), need += excess[v];
            if (excess[v] < 0) d.addEdge(v, T, -excess[v]);
        }
        return d.maxflow(S, T) == need;
    }
    ll maxFlow(int s, int t) {                  // -1 if infeasible
        int back = d.addEdge(t, s, LLONG_MAX / 4);
        if (!circulation()) return -1;
        ll base = d.e[back ^ 1].cap;            // flow on t->s
        d.e[back].cap = d.e[back ^ 1].cap = 0; // delete t->s
        return base + d.maxflow(s, t);
    }
};
```

Pitfalls

- Hungarian with `int` costs: potentials can exceed the cost range → use `long long`
- Hungarian needs n ≤ m; "forbidden" cells with INF costs overflow in `a - u - v`
- Blossom: resetting `base`, `par`, `used` **per search**; forgetting `match[v] == to` skip
- Simplex on unscaled data (1e9 and 1e−3 together) → wrong pivots
- Simplex result is fractional; rounding is only valid for TU matrices
- Lower bounds: forgetting the t → s edge, or checking saturation of the wrong side
- Reading off flows: add back l_i

Practice problems

- CSES 2129 *Task Assignment* (Hungarian)
- Library Checker *Assignment Problem*, *Matching on General Graph*
- UOJ #79 *一般图最大匹配* (general matching)
- Codeforces 375E *Red and Black Tree* (LP / simplex)
- Codeforces 704D *Captain America* (flow with lower bounds)
- LOJ #115, #116, #117 (feasible circulation, max flow and min flow with bounds)
- WF 2015 *Catering* (min-cost flow, Kattis)

Summary

- Hungarian = Dijkstra on reduced costs + dual potentials, $O(n^2m)$, ~30 lines
- Blossom = BFS with odd-cycle contraction via `base[]`, $O(V^3)$
- LP duality explains flows, matchings and potentials, and suggests reformulations
- Two-phase simplex solves small LPs; mind precision and integrality
- Lower bounds reduce to a max-flow saturation check on S', T'